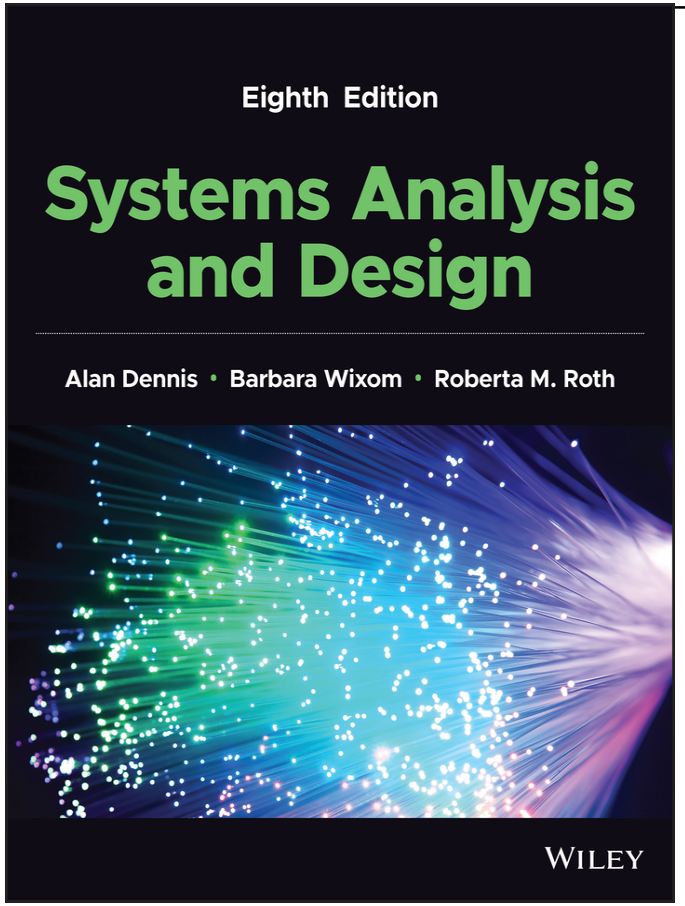
Software Development Lifecycles

Part 2



Systems Analysis and Design

Chapter 4

Understanding Processes with Use Cases and Process Models

Objectives

- Understand and Creating an use-case.
- Design Data Flow Diagram (DFD)
- Integrate Usecase - ERD - DFD

USE CASES

- **Definition:** A use case represents how a system interacts with its environment by illustrating user activities and system responses.

- **Purpose**: Captures functional requirements in a format that is easy for users and developers to understand.

- **Key Features**: Event-driven modeling, user-focused design, and integration into process models.

Elements of a Use Case

Basic Information

- **Name**: Descriptive title of the use case
- **ID**: Unique identifier (e.g., UC-1)
- **Description**: Brief summary of what the use case accomplishes
- **Priority**: Relative importance (High, Medium, Low)
- **Actor**: Person, system, or device that interacts with the system
- **Trigger**: The event that causes the use case to begin

Elements of a Use Case (continued)

Additional Components:

- **Preconditions**: State the system must be in before the use case begins
- **Normal Course**: Step-by-step process of the main scenario
- **Alternative Courses**: Branches in logic leading to successful completion
- **Postconditions**: Final state of the system after use case completion
- **Exceptions**: Error conditions that may occur during execution

Example

Use Case Name: Withdraw Cash from ATM	ID: UC-1	Priority: High
Actor: Bank Customer		
Description: Customer withdraws cash from their bank account.		
Trigger: Customer selects "Withdraw Cash" option at ATM.		
Type: ☑ External ☐ Temporal		
Preconditions: 1. Customer has a valid bank card. 2. Customer's account is active. 3. ATM is operational and has sufficient cash available.		
Normal Course 1. Customer inserts card into the ATM. 2. System validates the card. 3. System prompts for PIN. 4. Customer enters PIN. 5. System verifies the PIN against the bank system. 6. System displays account selection options (e.g., checking or savings). 7. Customer selects an account for withdrawal. 8. System displays available withdrawal amounts (e.g., $20, $50, $100). 9. Customer selects or enters a withdrawal amount. 10. System verifies sufficient funds are available in the account. 11. System dispenses the requested cash amount to the customer. 12. System prints a receipt if requested by the customer. 13. System returns the customer's card. 14. System returns to the welcome screen.		

Preconditions

- **Definition**: Preconditions define the state the system must be in before the use case begins.

- ATM Example Preconditions:

1. Customer has a valid bank card.
2. Customer's account is active and accessible via the ATM network.
3. ATM is operational and has sufficient cash available.

Normal Course

- **Definition**: The normal course lists the major steps performed to execute an event successfully, including inputs and outputs at each step.

Normal Course

Customer inserts card into the ATM
→ System validates card
→ System prompts for PIN
→ Customer enters PIN
→ System verifies PIN
→ System displays account options
→ Customer selects account
→ System displays withdrawal amounts
→ Customer selects or enters amount
→ System verifies sufficient funds
→ System dispenses cash
→ System prints receipt (optional)
→ System returns card
→ System returns to welcome screen.

Example (Continue)

Alternative Courses

- **8a: Customer selects "Other Amount" option:**
  1. **System prompts for manual amount entry by the customer.**
  2. **Customer enters desired withdrawal amount.**
  3. **Return to step 10 of the normal course.**

Postconditions

1. **The customer's account is debited by the withdrawal amount plus any applicable fees.**
2. **The ATM's cash inventory is reduced by the dispensed amount.**
3. **The transaction is logged in the banking system for auditing purposes.**

Exceptions

1. **Card is invalid:**
   - **The system retains or returns the card based on bank policy and displays an error message.**
2. **Incorrect PIN:**
   - **After three failed attempts, the system retains the card and notifies the customer of a security lockout.**
3. **Insufficient funds in account:**
   - **The system displays an error message and offers alternative withdrawal options (e.g., a smaller amount).**
4. **ATM has insufficient cash:**
   - **The system displays an error message indicating the maximum available withdrawal amount.**

Alternative Courses

- **Definition**: Alternative courses depict branches in logic that also lead to successful completion of the use case.

Alternative Courses

1. Customer selects "Other Amount" option:
    1. System prompts for manual entry of amount.
    2. Customer enters desired amount.
    3. If insufficient funds:
        1. Return to step 10 of normal course (funds verification).
2. Insufficient funds in account:
    1. Display error message and return to previous step.

Postconditions

- **Definition**: Postconditions define the final state of the system after completing the use case.

Postconditions

1. The customer's account is debited by the withdrawal amount plus any applicable fees.
2. The ATM's cash inventory is reduced by the dispensed amount.
3. The transaction is logged in the banking system for auditing purposes.

Exceptions

- **Definition**: Exceptions describe error conditions or unusual occurrences that prevent successful completion of the use case.

Exceptions

1. Invalid Card:
    1. Display error message; retain or return card based on bank policy.
2. Incorrect PIN:
    1. Prompt for re-entry; retain or return card after multiple failed attempts.
3. Insufficient Cash in ATM:
    1. Display error message indicating maximum available withdrawal amount.

Summary of Use Case Components

1. Basic Information captures key details like name, actor, and trigger.
2. Preconditions ensure readiness before execution.
3. Normal Course defines steps for success.
4. Alternative Courses handle variations leading to success.
5. Postconditions describe final states after execution.
6. Exceptions address error conditions.

Additional Use Case Issues

Organizations may include additional sections in use case forms, such as:

- **Alternative Paths:** Variations in logic leading to success.
- **Summary of Inputs/Outputs:** Key data exchanged during the use case.
- **Frequency of Use:** How often the use case is executed.
- **Business Rules:** Policies or constraints affecting the use case.
- **Special Requirements:** Technical or operational needs.
- **Assumptions:** Conditions assumed to be true for execution.
- **Notes and Issues:** Additional observations or concerns.

Example of Additional Use Case Sections

ATM Withdrawal Example:

- **Frequency of Use**: High (daily transactions).
- **Business Rules:** PIN must be encrypted per PCI-DSS standards.
- **Special Requirements:** ATM must support multi-language interfaces.
- **Assumptions**: ATM network connectivity is stable.

Chain of Use Cases

- Complex systems often involve multiple interconnected use cases.
- Example:
  - "Withdraw Cash" → "Check Balance" → "Deposit Cash."
- Each use case has preconditions and postconditions that link them logically.

Alternative Use Case Formats

1. Full-Dressed Use Case:
   - Highly detailed and structured format.
   - Suitable for complex systems with strict documentation needs.
2. Casual Use Case:
   - Informal and less detailed format.
   - Used for simpler systems or early-stage analysis.

Use Cases and Functional Requirements

- Use cases provide a user-centric view of system requirements.
- Functional requirements translate user actions into developer specifications.

- Example Transformation:
  - User Story: "Withdraw $100 quickly."
  - Functional Requirement: Validate card authenticity within 3 seconds; encrypt PIN using AES-256.

Functional Requirements Derived from ATM Example

1. System must validate card authenticity within a maximum of 3 seconds.
2. PIN encryption must comply with PCI-DSS standards.
3. ATM must display available withdrawal amounts dynamically based on account balance.

Use Cases and Testing

- Use cases help define test scenarios early in development.
- Benefits:
  - Early identification of test elements ensures thorough coverage.
  - Testing personnel can prepare detailed test cases in advance.

Testing Derived from ATM Example

Test Plan:

1. Input: Valid card, correct PIN, sufficient funds.
2. Expected Output: Cash dispensed, receipt printed, transaction logged.

Error Handling Tests:

1. Invalid Card → System displays error message, retains or returns card based on policy.
2. Incorrect PIN → System prompts for re-entry, locks card after multiple failed attempts.

Steps to Create a Use Case

1. Identify the triggering event (e.g., "Withdraw Cash").
2. Develop a list of major steps (e.g., card validation).
3. Identify inputs (e.g., PIN) and outputs (e.g., cash dispensed).

Step-by-Step Process for Creating a Use Case

1. Define the basic information:
    1. Name, ID, Actor, Trigger, Description.
2. Specify preconditions:
    1. State required before execution (e.g., valid card).
3. Outline normal course steps:
    1. Logical sequence of actions leading to success.

Adding Alternative Courses

1. Identify variations that lead to successful completion:
   - Example: Customer selects "Other Amount."
2. Specify logic for handling edge cases:
   - Example: Insufficient funds → Return to previous step.

Defining Postconditions

1. Describe the final state after execution:
   - Example: Account debited by withdrawal amount; transaction logged.
2. Ensure alignment with system goals:
   - Example: Audit trail maintained for regulatory compliance.

Handling Exceptions

1. Define error conditions and responses:
   - Invalid Card → Display error message; retain or return card.
2. Ensure system resilience to unusual occurrences:
   - Insufficient Cash → Display maximum available withdrawal amount.

Role-Playing to Validate Use Cases

1. Engage users in role-playing exercises to verify accuracy.
2. Example Scenario:
   - User simulates ATM withdrawal; team observes system responses.

Creating Use Cases

Step	Activities	Typical Questions Asked[a]
1. Identify the use cases.	Start a use case report form for each use case by filling in the name, description, and trigger. If there are more than nine use cases, group them into packages.	Ask who, what, when, and where about the use cases (or tasks). What are the major tasks that are performed? What triggers this task? What tells you to perform this task?
2. Identify the major steps within each use case.	For each use case, fill in the major steps needed to complete the task.	Ask how about each use case. What information/forms/reports do you need to perform this task? Who gives you these information/forms/reports? What information/forms/report does this produce and where do they go? How do you produce this report? How do you change the information on the report? How do you process forms? What tools do you use to do this step (e.g., paper, e-mail, phone)?
3. Identify elements within steps.	For each step, identify its triggers and its inputs and outputs.	Ask how about each step. How does the person know when to perform this step? What forms/reports/data does this step produce? What forms/reports/data does this step need? What happens when this form/report/data is not available?
4. Confirm the use case.	For each use case, validate that it is correct and complete.	Ask the user to execute the process, using the written steps in the use case—that is, have the user role-play the use case.

SUMMARY

- A use case contains all the information needed to build one part of a process model, expressed in an informal, simple way.
- When writing a use case:

1. Identify triggering events and actors.
2. Develop major steps with inputs/outputs.
3. Add alternative courses and exceptions.
4. Define preconditions and postconditions.
5. Validate through user role-playing.

Data Flow Diagrams (DFDs)

Data Flow Diagrams

- **Definition**: A DFD is a graphical representation of the flow of data through a system.
- **Purpose**: Helps analyze and model business processes by illustrating how data moves between processes, data stores, and external entities.
- **Two types:**
  - **Logical DFD**: Focuses on business processes without technical details.
  - **Physical DFD**: Includes implementation details like hardware and software.

Data Flow Diagrams

Reading Data Flow Diagrams

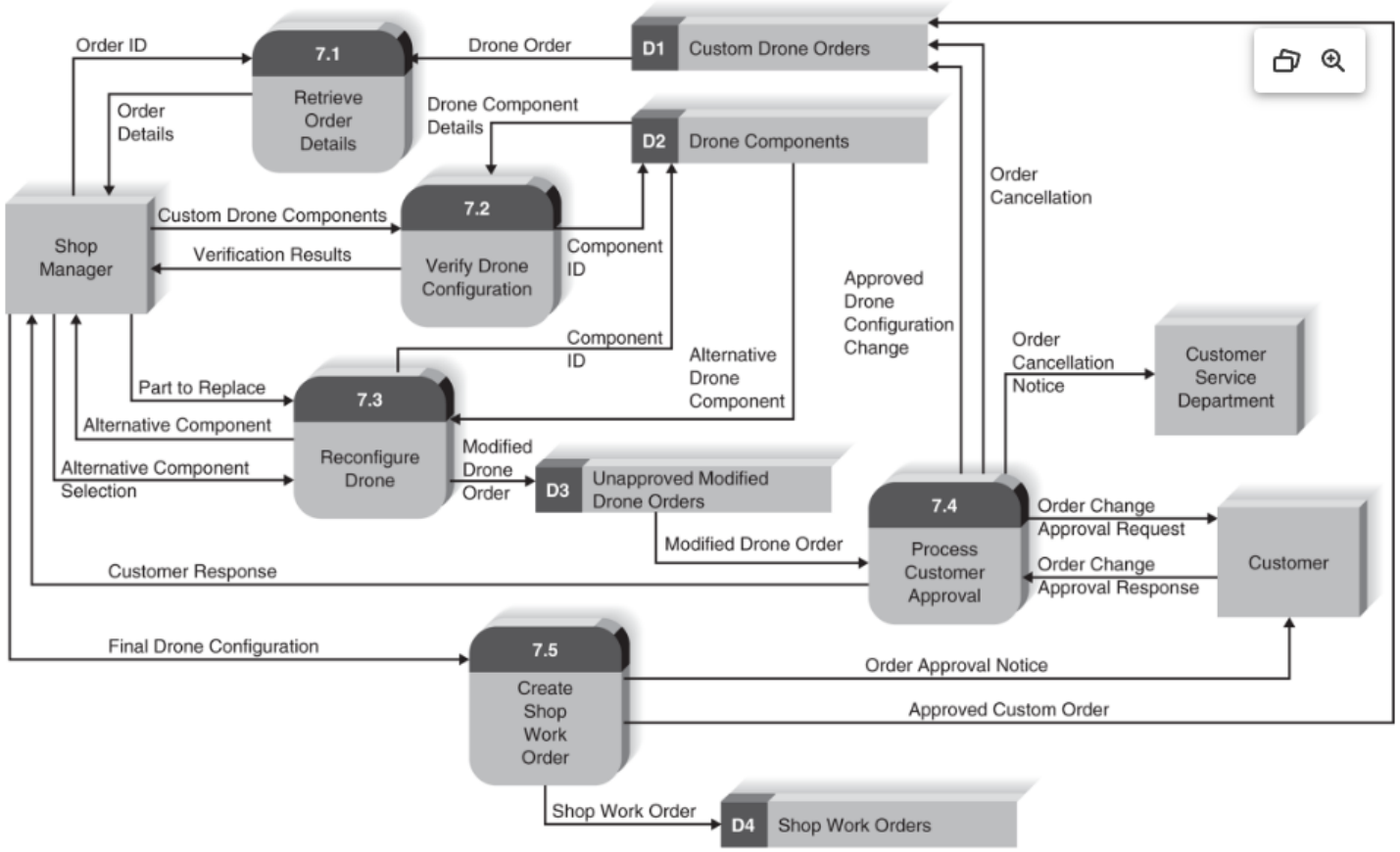


FIGURE 4-12 Shop manager approval of custom drone order level 1 DFD.

Components of a DFD

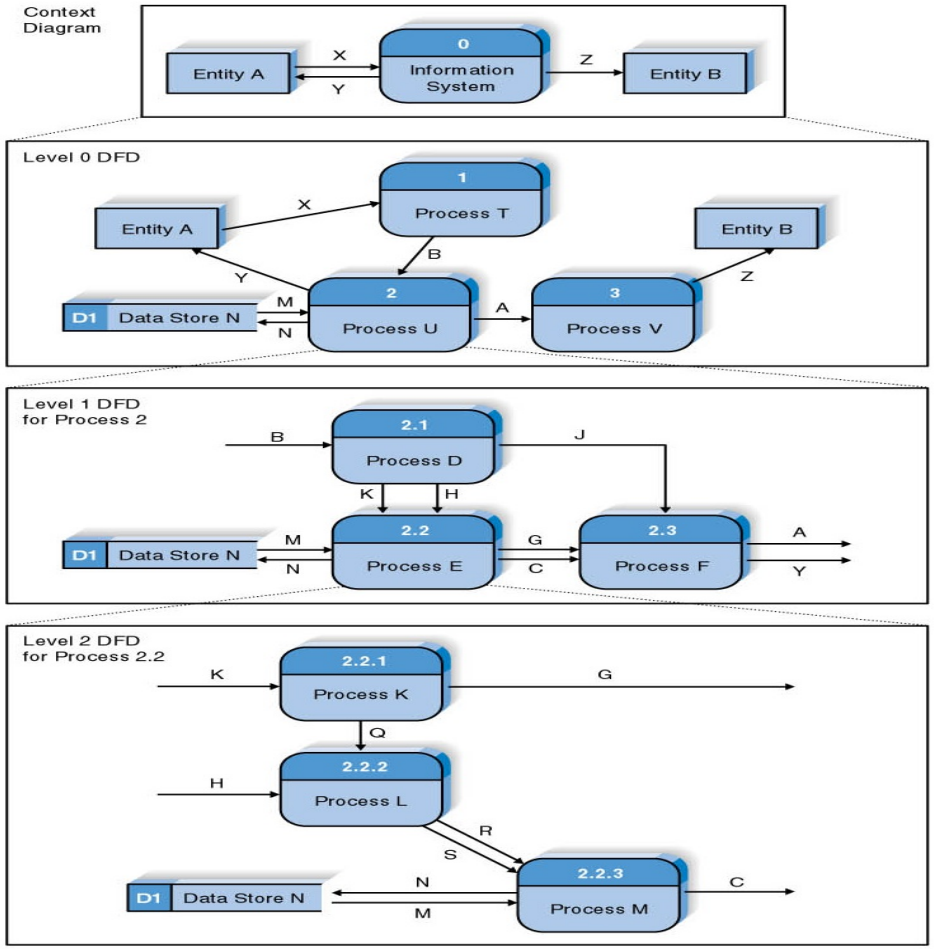
- **Processes**: Represent activities or functions performed for specific business reasons.
- **Data Flows**: Show the movement of data between processes, data stores, and external entities.
- **Data Stores**: Represent collections of stored data (e.g., databases).
- **External Entities**: Represent people or systems outside the scope of the system being modeled.

Components of a DFD

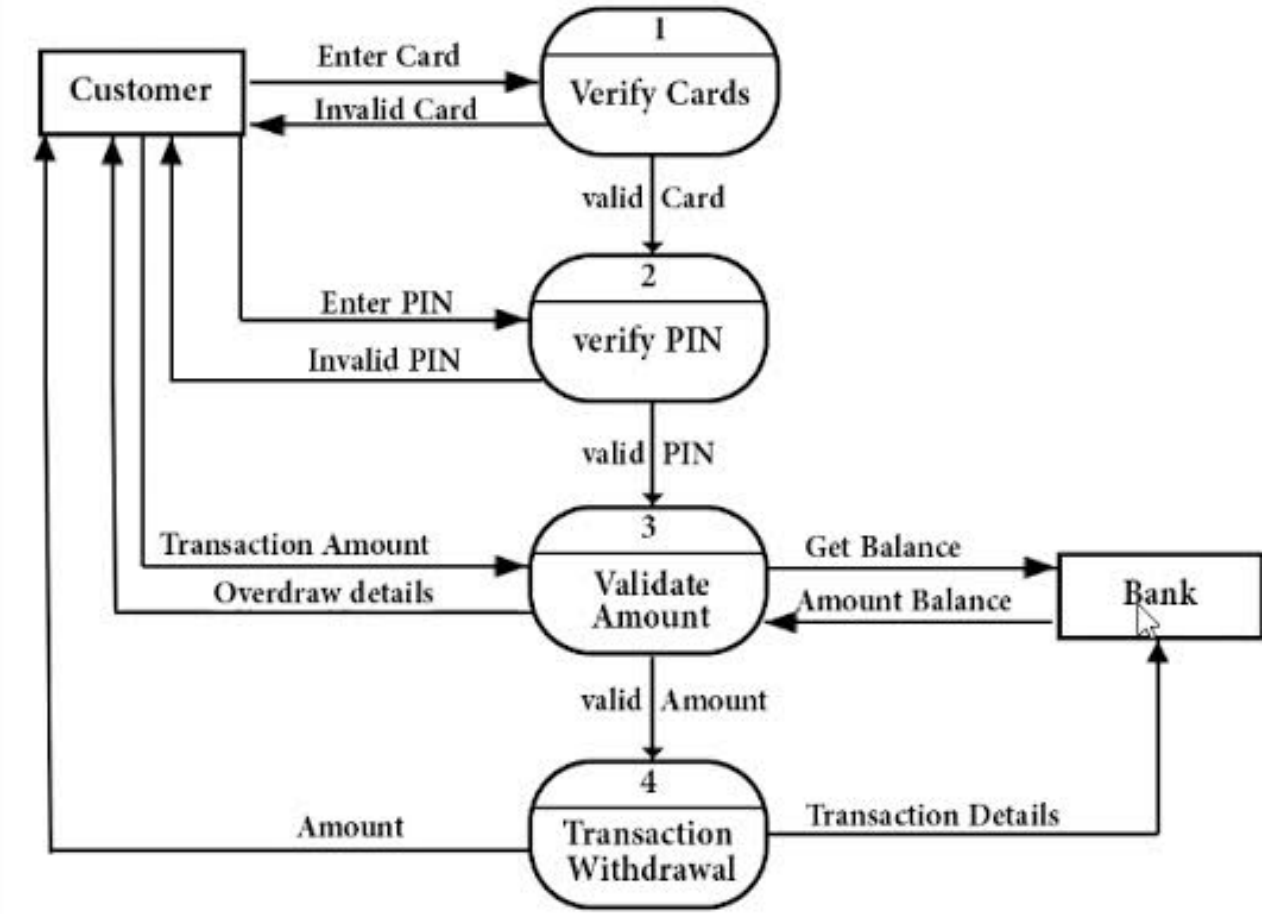
Component	Symbol	Description
Process	Circle or Rectangle	Represents an activity or function.
Data Flow	Arrow	Shows movement of data between elements.
Data Store	Open Rectangle	Represents stored data.
External Entity	Rectangle	Represents external systems or actors.

Data Flow Diagram Element	Typical Computer-Aided Software Engineering Fields	Gane and Sarson Symbol	DeMarco and Yourdon Symbol
Every *process* has a number a name (verb phase) a description at least one output data flow at least one input data flow	Label (name) Type (process) Description (what is it) Process number Process description (structured English) Notes	1 Name	Name
Every *data flow* has a name (a noun) a description one or more connections to a process	Label (name) Type (flow) Description Alias (another name) Composition (description of data elements) Notes	Name →	Name →
Every *data store* has a number a name (a noun) a description one or more input data flows one or more output data flows	Label (name) Type (store) Description Alias (another name) Composition (description of data elements) Notes	D1 Name	D1 Name
Every *external entity* has a name (a noun) a description	Label (name) Type (entity) Description Alias (another name) Entity description Notes	Name	Name

(cont'd)


Context Diagram
Entity A
X
Y
0
Information System
Z
Entity B
Level 0 DFD
Entity A
X
1
Process T
Entity B
B
Y
Z
D1
Data Store N
M
N
2
Process U
A
3
Process V
Level 1 DFD for Process 2
B
2.1
Process D
J
K
H
D1
Data Store N
M
N
2.2
Process E
G
C
2.3
Process F
A
Y
Level 2 DFD for Process 2.2
K
2.2.1
Process K
G
Q
H
2.2.2
Process L
R
S
D1
Data Store N
N
M
2.2.3
Process M
C

Using DFDs to Define Business Processes



Context Diagram

- A high-level overview showing the entire system as a single process interacting with external entities.
- Includes inputs and outputs but excludes detailed processes or data stores.

Example Context Diagram – ATM System

- **Process Name:** ATM Withdrawal System (ID: Process 0)
- **External Entities:** Customer, Banking Network.
- **Data Flows:** Card Input, PIN Input, Withdrawal Request, Cash Output.

Level 0 Diagram – Definition

- Breaks down the system into major high-level processes and shows their interconnections.
- Includes processes, data flows, external entities, and data stores.

Example Level 0 Diagram – ATM Withdrawal

Process ID	Process Name	Data Store
P1	Validate Card	Card Validation Store
P2	Verify PIN	Account Database
P3	Dispense Cash	Transaction Log

Data Flows:
Card Input
→ Validate Card
→ PIN Input
→ Verify PIN
→ Withdrawal Request
→ Dispense Cash
→ Cash Output.

Level 1 Diagram – Definition

- Decomposes each process from the Level 0 diagram into subprocesses for more detail.
- Ensures that all inputs and outputs from Level 0 are accurately represented.

Exercise: Level 1 Diagram - PIN Verification Process

Parent Process: Verify PIN (P2)
Subprocesses:

- Decrypt Card Data → Retrieve Hashed PIN → Compare Hash Values → Send Authentication Status.

Data Flows:

- Encrypted PIN Input → Decrypt Card Data → Authentication Response Output.

Principles of Decomposition

1. Break down complex processes into smaller subprocesses for clarity.
2. Ensure consistency across levels (balancing).
3. Each process should have at least one input and one output.

Level 2 Diagrams

- The Level 2 DFD further decomposes a single process from the Level 1 DFD into more granular subprocesses, showing detailed interactions between data flows, data stores, and external entities.

Create DFD Level 2 - Decompose P3

Process Withdrawal from the Level 1 DFD into its subprocesses.

Exercise: Create DFD Level 2 - Decompose P3

Processes:

P3.1: Verify Account Balance

- Checks if the account has sufficient funds for the requested withdrawal amount.

P3.2: Deduct Amount from Account

- Updates the account balance by deducting the withdrawal amount plus applicable fees.

P3.3: Dispense Cash

- Handles the physical dispensing of cash to the customer.

P3.4: Log Transaction

- Records transaction details in the Transaction Log for auditing purposes.

Exercise: Create DFD Level 2 - Decompose P3

Data Stores:

- D2: Account Database
  - Stores account information, including balances and transaction history.
- D3: Transaction Log
  - Maintains a record of all completed transactions for auditing.

External Entities:

- Customer
  - Provides inputs (withdrawal request) and receives outputs (cash).

Data Flows:

1. Withdrawal Request → Verify Account Balance → Sufficient Funds Status
2. Sufficient Funds Status → Deduct Amount from Account → Updated Account Balance
3. Updated Account Balance → Dispense Cash → Cash Output
4. Transaction Details → Log Transaction → Audit Record

Summary – Creating DFDs

Steps to Create DFDs:

1. Build the context diagram to define system boundaries.
2. Create DFD fragments based on use cases.
3. Combine fragments into a Level 0 diagram.
4. Develop Level 1 diagrams for each major process.
5. Validate diagrams for completeness and correctness.

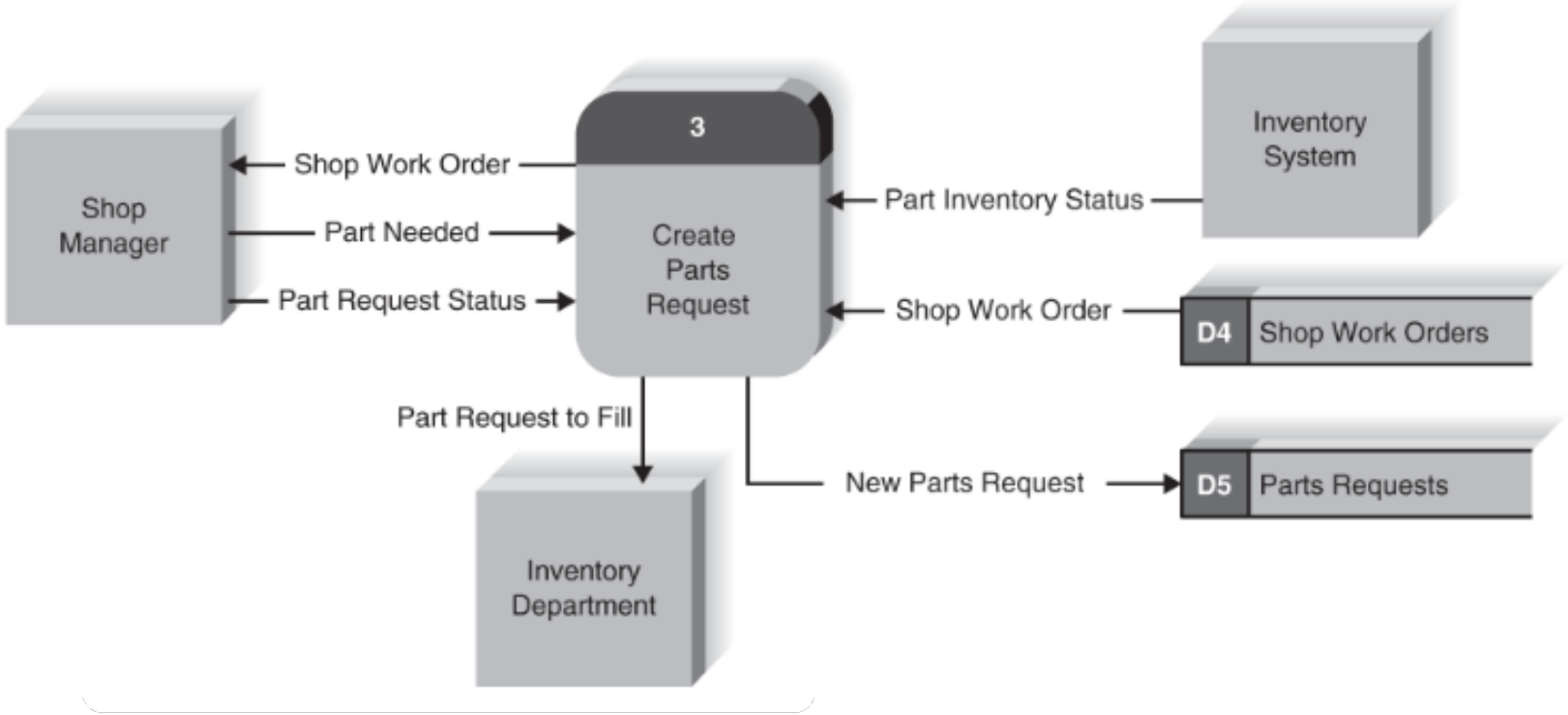
DFD Fragments

- **Definition**:
  A DFD fragment represents a part of a system's process model, typically derived from a single use case. It includes processes, data flows, external entities, and data stores relevant to that use case.
- **Purpose**:
  - Simplifies complex systems by breaking them into smaller, manageable pieces.
  - Ensures clarity and focus on individual processes.
  - Serves as the foundation for creating Level 0 and Level 1 DFDs.
- **Key Features:**
  - Each fragment corresponds to one use case.
  - Includes inputs, outputs, and related data flows.

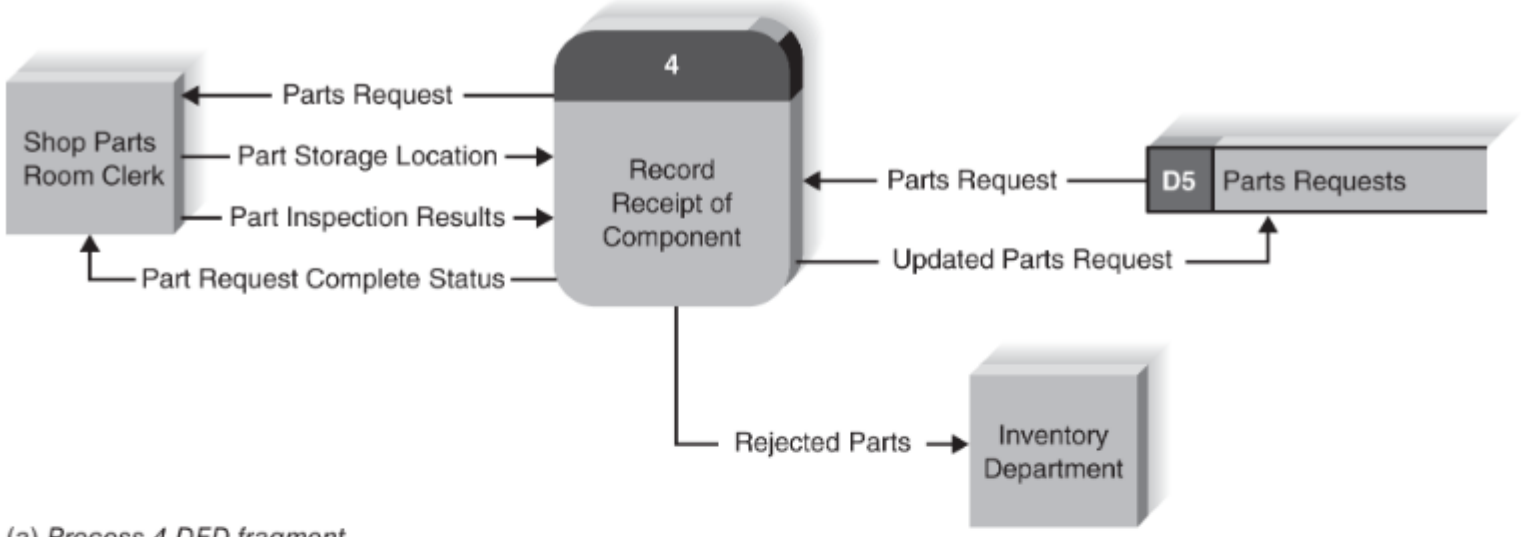
Example of Fragment

Use Case Name: Create Parts Request	**ID:** UC-3	**Priority:** High
Actor: Shop Manager		
Description: This use case describes how the Shop Manager creates a Parts Request		
Trigger: Shop Manager receives notice of new shop work order arrival from Sales System		
Type: ☑ External ☐ Temporal		

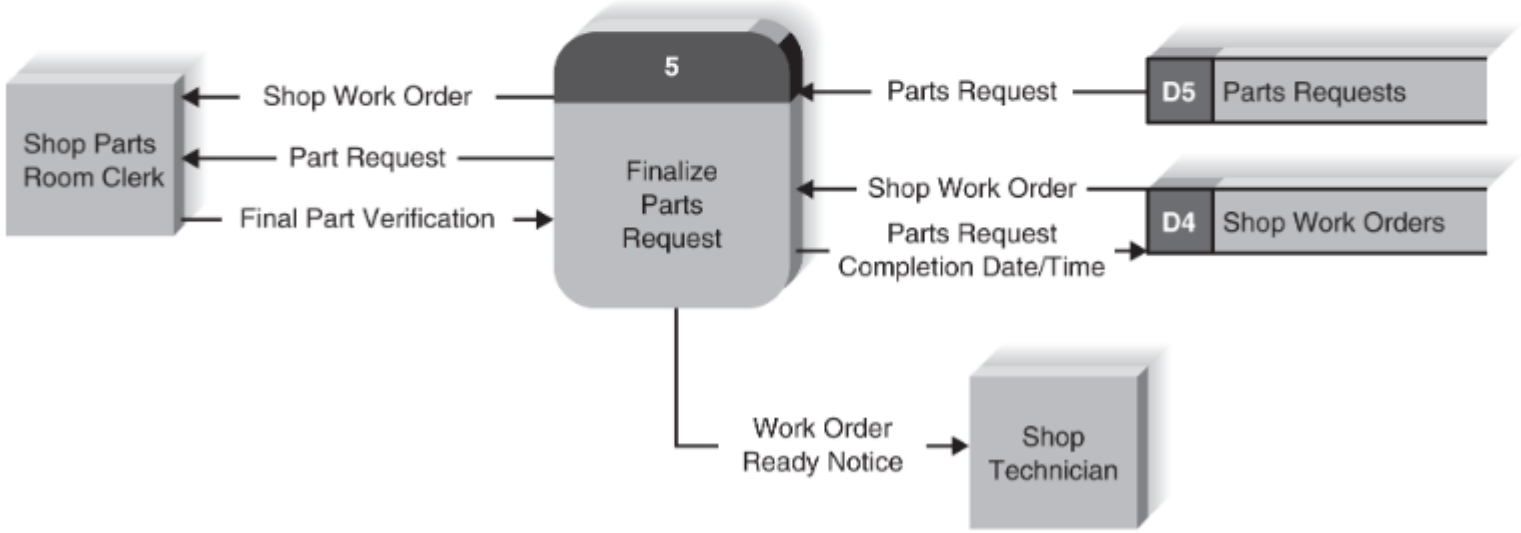
Summary Inputs	Source	Summary Outputs	Destination
Part ID and quantity Part inventory status Part request status Shop work order	Shop manager Inventory System Shop manager Shop work order data store	Shop work order New part request record Parts Request to fill	Shop manager Parts Request datastore Inventory department



Additional Example of Fragment



(a) *Process 4 DFD fragment*



(b) *Process 5 DFD fragment*

FIGURE 4-19 Additional DFD fragments for DrōnTeq customization shop management system.

How to Combine DFD Fragments

1. Identify all use cases in the system.
2. Create a DFD fragment for each use case.
    1. Include relevant processes, data flows, data stores, and external entities.
3. Combine fragments into a Level 0 DFD:
    1. Ensure all fragments are connected logically.
    2. Balance inputs and outputs across fragments.
    3. Avoid redundancy in processes or data flows.

Entity Relationship Diagram (ERD) - Remind

- **Definition**:
  An ERD is a visual representation of the data created, stored, and used by a business system. It focuses on the relationships between entities and their attributes.
- **Key Components:**
  - **Entity**: Represents a person, place, event, or thing about which data is collected (e.g., Customer, Account).
  - **Attribute**: Information captured about an entity (e.g., Name, Account Balance).
  - **Relationship**: Association between entities (e.g., Customer owns Account).
  - **Cardinality**: Specifies the number of instances in a relationship (e.g., 1:N, M:N).
  - **Modality**: Defines whether relationships are mandatory or optional (e.g., Null or Not Null).

Comparison Between DFD and ERD

Feature	Data Flow Diagram (DFD)	Entity Relationship Diagram (ERD)
Focus	Processes and data flows	Data structure and relationships
Purpose	Models how data moves through the system	Models how data is stored and organized
Key Components	Processes, Data Flows, Data Stores, External Entities	Entities, Attributes, Relationships
Representation	Dynamic flow of data	Static organization of data
Use Case Alignment	Derived from process steps	Derived from inputs/outputs and data stores

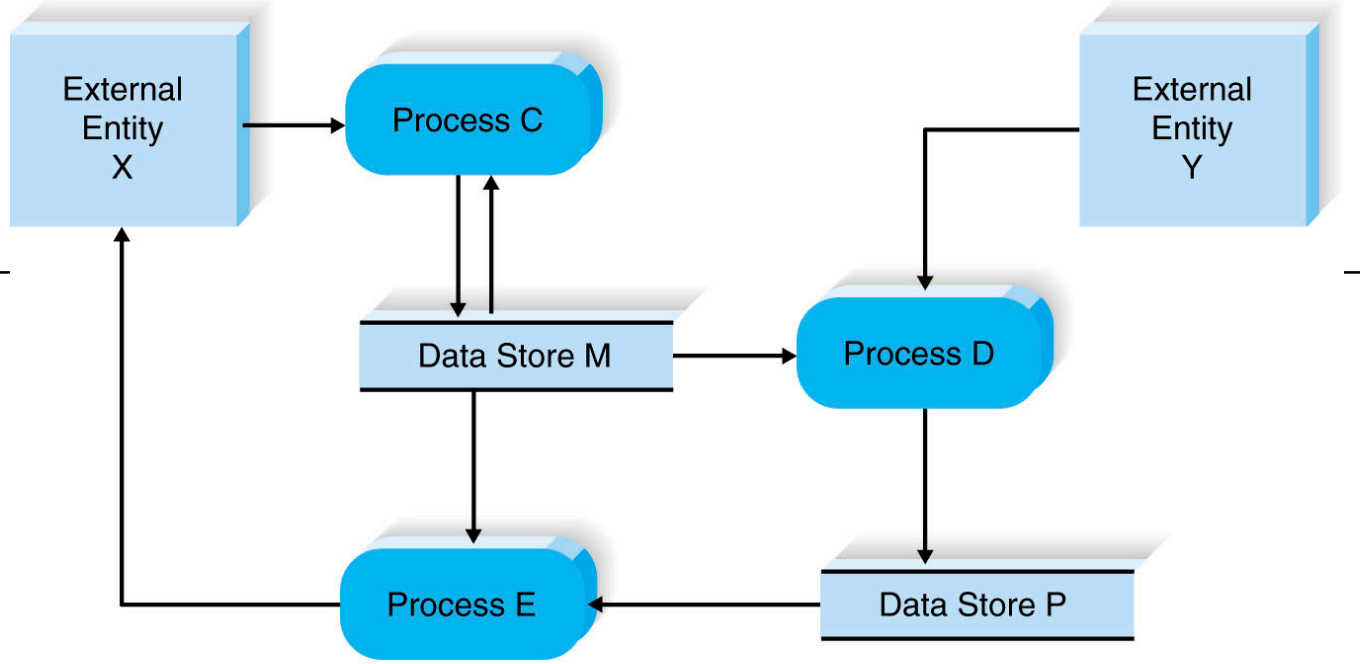
Integrating DFD and ERD

- **Why Combine?**
  - DFDs focus on processes, while ERDs focus on data organization. Together, they ensure both workflows and data structures are aligned.

Integrating DFD and ERD

- Steps to Integrate:
  1. Identify all data stores in the DFD.
     - Example: "Account Database" in the ATM system.
  2. Map each data store to an entity in the ERD.
     - Example: "Account Database" ↔ Account entity.
  3. Validate that all attributes in the ERD correspond to data flows in the DFD.
     - Example: "Withdrawal Request" ↔ Transaction attributes.
  4. Use CRUD Matrix:
     - Ensure processes in the DFD align with operations on entities in the ERD (Create, Read, Update, Delete).

Integrating DFD and ERD



	Process C	Process D	Process E
Data Entity M			
Attribute M-1	CRUD	R	R
Attribute M-2	CRUD		R
Attribute M-3	CRUD	R	
Attribute M-4	CRUD		R
Data Entity P			
Attribute P-1		C	R
Attribute P-2		C	
Attribute P-3		C	R

Example Integration – ATM Withdrawal System

From DFD:

- Data Store: Account Database → Used by processes like "Verify PIN" and "Process Withdrawal."

To ERD:

- Entity: Account → Attributes like AccountID and Balance.

Process	Create	Read	Update	Delete
Validate Card		X		
Verify PIN		X		
Process Withdrawal	X	X	X	

Example Integration – ATM Withdrawal System

Design ATM withdraw diagrams:

- Use-case
- ERD
- DFD